Blockchain / Battery Passport / Compliance Layer

Development, Integration, Verification, Testing and Acceptance Guide for the Battery Thermal Intelligence Cloud Platform

Scope: Step 6 of the supplied architecture: high-volume off-chain storage, periodic SHA-256/Merkle anchoring, Hyperledger Fabric / chaincode, battery-passport ledger, provenance, compliance notarization, smart-contract access control and anchored critical events.

Purpose: Convert the Step 6 architecture into an engineering workflow that can be implemented, tested, benchmarked, audited and accepted for an OEM/fleet/battery-manufacturer SaaS platform.

Design principle: Do not put raw battery telemetry on-chain. Keep high-volume, sensitive and mutable records in governed off-chain storage and use the permissioned ledger as a tamper-evident system of proof, provenance and controlled multi-party state.

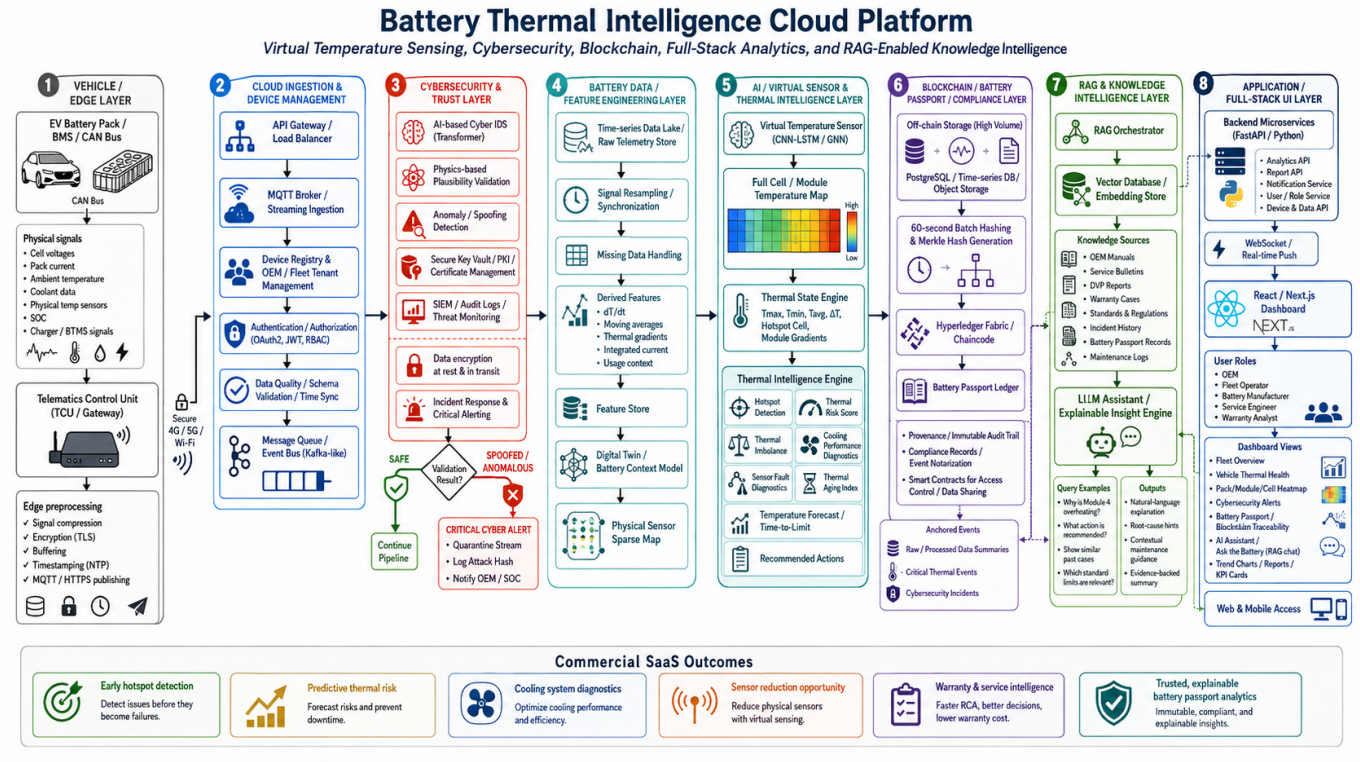
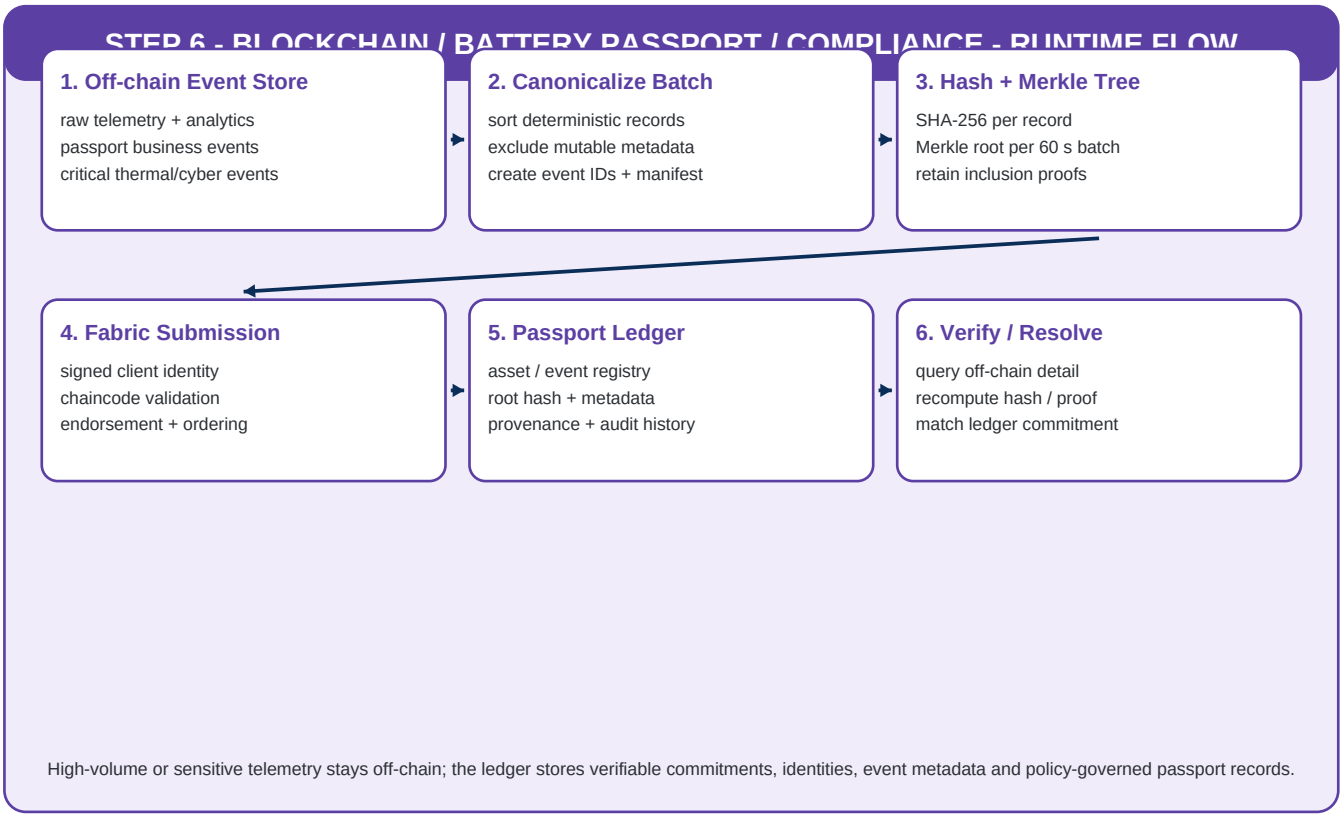


Figure 1. Source platform architecture. This guide expands Step 6 - Blockchain / Battery Passport / Compliance Layer.

1. What Step 6 Must Achieve

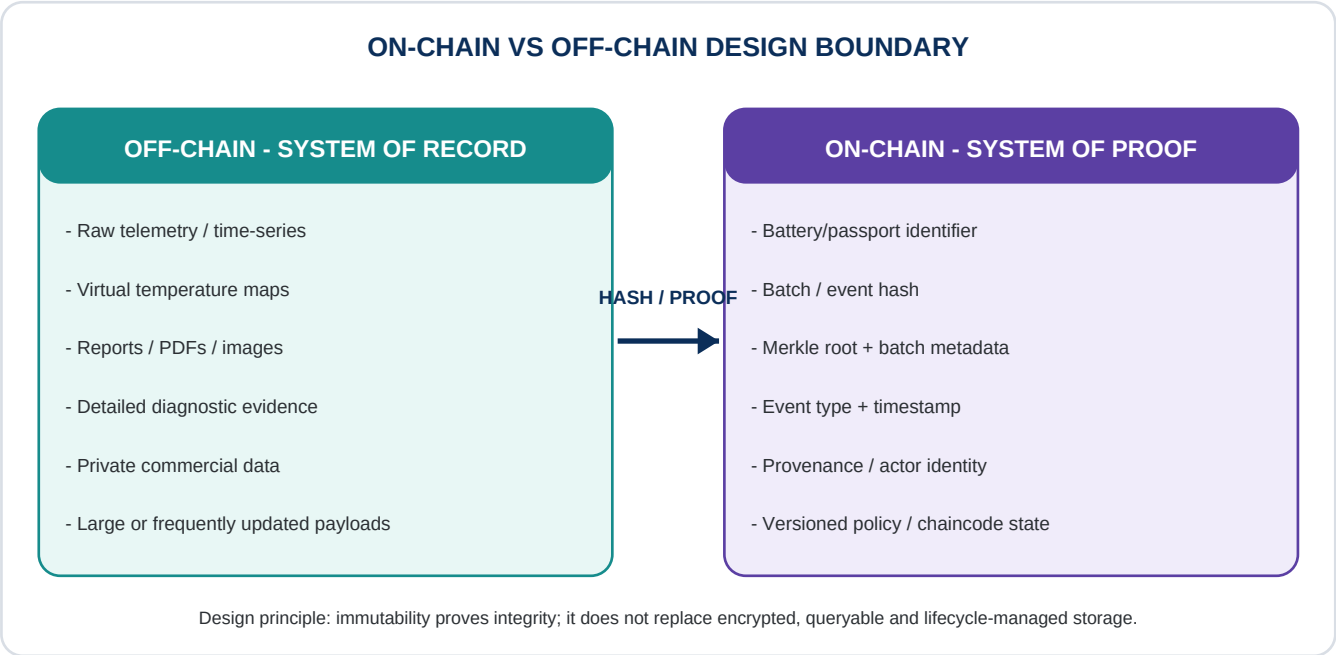
Step 6 must create verifiable battery lifecycle records without turning blockchain into the primary telemetry database. It must prove who recorded an event, what data version was committed, whether later data still matches that commitment, and which organizations are authorized to read or update each passport domain.



Objective	What Step 6 must provide	Primary mechanism
Integrity proof	Detect alteration of off-chain records after commitment	SHA-256 record hashes + Merkle root anchoring
Provenance	Identify actor, organization, asset, time and lifecycle event	Fabric MSP identity + signed transaction + immutable history
Multi-party trust	Avoid one organization unilaterally rewriting shared lifecycle state	Permissioned peers, endorsement policy, ordering service
Confidentiality	Restrict commercial or personal data to authorized organizations	Off-chain access control + Fabric private-data collections / channels
Compliance evidence	Retain auditable battery-passport declarations and events	Versioned ledger entries + document hashes + source references
Lifecycle interoperability	Exchange standardized identifiers/ events across parties	Battery Pass data model + GS1 EPCIS / Digital Link where applicable
Operational auditability	Reconstruct critical thermal/ cyber/ passport events	event IDs, timestamps, model/ rule versions, anchored summaries

2. On-Chain / Off-Chain Boundary

The most important architecture decision is the data boundary. Blockchain should preserve a compact, deterministic proof and shared business state; databases/object stores should preserve the detailed payload needed for analytics, dashboards, reports, retention management and privacy handling.



Design principle: immutability proves integrity; it does not replace encrypted, queryable and lifecycle-managed storage.

Recommended decision rules

Data category	Store off-chain?	Anchor / store on-chain?	Reason
100 ms / 1 s battery telemetry	Yes	Merkle root only	Too large and frequently updated for ledger storage
Virtual temperature maps / AI outputs	Yes	Selected summary/ event hash	Retain detail for analytics while preserving audit proof
Critical thermal event	Yes	Yes: event ID, severity, root/ hash, time, source version	High-value evidence for warranty/ safety review
Cybersecurity incident	Yes	Yes: incident hash and immutable event metadata	Forensic traceability without exposing raw sensitive logs
Battery passport identity / lifecycle event	Yes for detail	Yes for shared business state / proof	Multi-party provenance and transfer-of-custody
Certificates / proprietary documents	Encrypted off-chain	Hash/ reference only	Avoid leakage and enable revocation/ retention policies
Personal data / driver identifiers	Prefer segregated off-chain	Do not place directly on-chain	Privacy and erasure obligations

Rule: if a field must later be corrected, deleted, access-restricted, or changes frequently, keep the authoritative value off-chain and anchor a versioned proof instead.

3. Detailed Runtime Workflow

3.1 Receive trusted events

Consume only events that have passed the Cybersecurity & Trust Layer, plus identity, trust status, source versions and timestamps. The blockchain service must not become a bypass around the trust gate.

3.2 Persist detailed off-chain record

Write raw/process data to PostgreSQL, time-series storage or object storage using immutable event IDs and a deterministic schema/version. Capture retention classification and tenant ownership.

3.3 Build a deterministic 60-second batch

At batch close, select eligible events, sort them by deterministic keys, normalize canonical JSON fields, exclude ephemeral fields, and produce a manifest containing batch ID, event count and schema version.

3.4 Hash records

Generate SHA-256 for each canonical event. Never hash non-deterministic serializations. Record the canonicalization algorithm and version.

3.5 Generate Merkle root and inclusion proofs

Build the Merkle tree from ordered leaf hashes. Persist the root plus sufficient proof material to verify any individual event later without re-reading unrelated records.

3.6 Submit Fabric transaction

The backend signs the transaction with its organization identity. Chaincode validates asset/passport ID, event type, signer role, idempotency key, timestamps and allowed state transition.

3.7 Endorse, order and commit

Required organizations endorse according to policy; the ordering service establishes transaction order; peers validate and commit the block/ledger state.

3.8 Update Battery Passport ledger

Store the commitment and any shared lifecycle state such as asset identity, event class, provenance, compliance statement reference, ownership/custody status and version.

3.9 Verify on demand

When a dashboard, auditor, service engineer or RAG workflow requests proof, retrieve the off-chain record, recompute the leaf hash/inclusion proof and compare against the committed Merkle root.

3.10 Monitor and reconcile

Continuously detect failed submissions, missing batch anchors, duplicate events, endorsement failures, lagging peers, certificate expiry, chaincode/version drift and mismatch between off-chain manifests and ledger commitments.

4. Battery Passport Event and Data Model

Use a lifecycle event model rather than a single monolithic passport document. A passport is the current resolved view of identity, declarations and lifecycle history, while each change remains individually traceable.

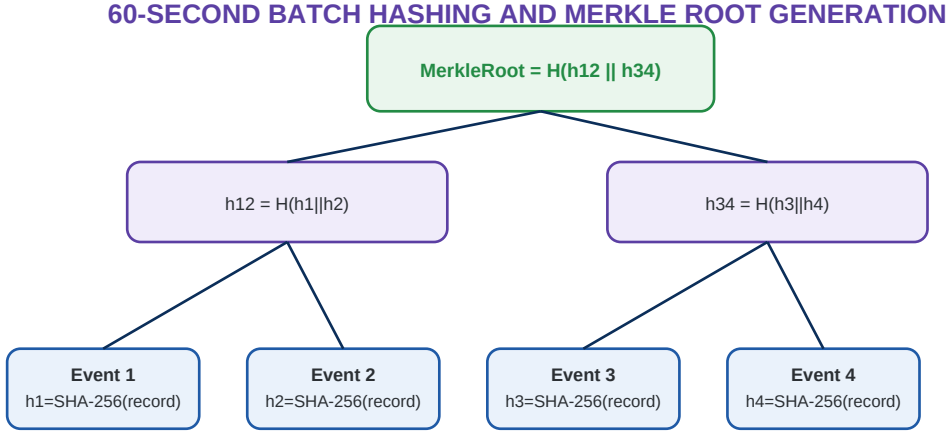
Event family	Typical event types	Key data elements	Ledger treatment
Identity / manufacturing	battery_created, pack_assembled, component_linked	battery_id, manufacturer, model, chemistry, date, facility	Shared passport state + source hash
Compliance	declaration_added, certificate_updated, carbon_footprint_versioned	document_id, issuer, scope, version, valid_from/ to	Hash/ reference + provenance + version
Use phase	vehicle_bound, service_event, ownership/ custody_change	asset IDs, actor org, timestamp, location class	Lifecycle event / custody state
Health / diagnostics	SOH_snapshot, thermal_event, degradation_summary	metric, method/ version, confidence, source batch	Summary/ hash; detailed data off-chain
Cybersecurity	security_incident, trust_violation	incident ID, class, severity, evidence hash	Event proof only; sensitive logs off-chain
Second life	inspection, grading, repurposed, module_reconfigured	grade, test method, capacity, new asset linkage	New/ derived passport relationship
End of life	collection, dismantling, recycling, material_recovery	operator, process, material fractions, certificates	Traceability + compliance proof

Minimum immutable identifiers

- battery_passport_id and physical product identifier
- event_id and idempotency_key
- organization MSP / actor identity
- event_time and commit_time
- schema_version and chaincode_version
- source_system and offchain_object_id
- content_hash / merkle_root / batch_id
- previous lifecycle state reference where required

5. 60-Second Batch Hashing and Merkle Anchoring

The 60-second interval shown in the platform is a design choice for reducing blockchain transaction volume. The batch duration should be calibrated against event criticality, throughput, cost, audit requirements and acceptable proof latency. Critical events may bypass the normal batch and be anchored immediately.



Persist: batch_id, first/last event time, count, schema version, root, signer, chaincode version.

Verification: recompute leaf hash -> reconstruct inclusion proof -> compare root with committed ledger value.

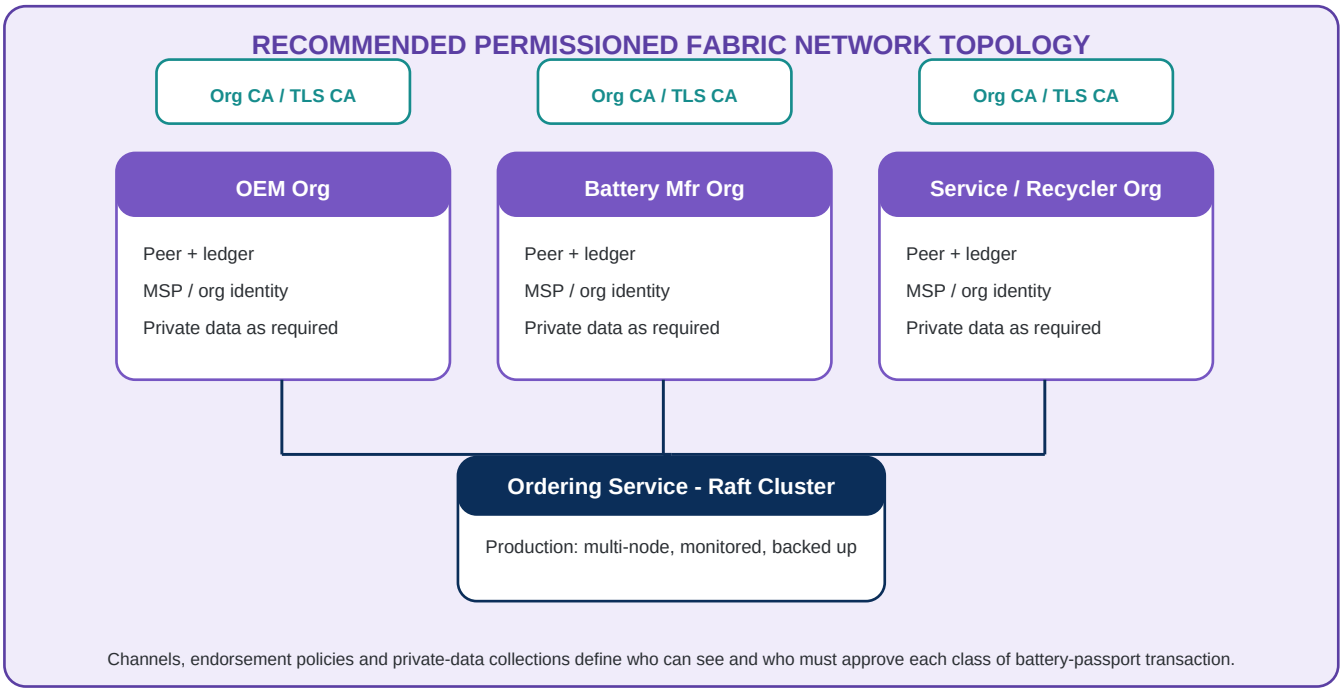
Canonicalization rules

Rule	Requirement
Stable field order	Canonical JSON or deterministic binary encoding; same logical record must always generate the same bytes
Stable numeric representation	Define decimal precision, units, NaN handling and sign convention before hashing
Time normalization	Use UTC and a fixed timestamp format/ precision
Exclude mutable transport fields	Do not hash request IDs, retry counters or ingestion metadata unless they are part of the declared evidence
Version everything	schema_version, canonicalization_version and hash_algorithm must accompany proof metadata
Duplicate protection	Use event_id + idempotency_key and reject duplicate anchor submissions

Critical requirement: acceptance tests must prove that one-bit modification of any anchored event changes the recomputed proof and fails ledger verification.

6. Hyperledger Fabric Network Architecture

For an OEM-facing battery passport, a permissioned consortium network is generally more appropriate than a public chain because participants are known organizations with contractual roles, privacy boundaries and endorsement responsibilities. Hyperledger Fabric supports organization identities, channels, private data and policy-based endorsement.



Channels, endorsement policies and private-data collections define who can see and who must approve each class of battery-passport transaction.

Recommended production building blocks

Component	Purpose	Production design consideration
Peer nodes	Host ledger, world state and chaincode execution	At least one highly available peer per participating organization; monitor block height and resource use
Ordering service	Establish total transaction order	Multi-node Raft cluster across failure domains; avoid single-node production orderer
Fabric CA / enterprise CA	Issue, renew and revoke organization identities	Separate enrollment and TLS trust as appropriate; protect CA keys/ HSM
Channel(s)	Partition shared ledger domains	Use only when legal/ business isolation requires separate ledgers; avoid unnecessary channel explosion
Private Data Collections	Share selected values only with authorized orgs	Define collection policy, dissemination, retention/ purge and reconciliation tests
CouchDB / world state	Queryable current state	Index only ledger state needed by chaincode/ app queries; full analytics remain off-chain
Gateway / backend service	Submit/ evaluate transactions	Use mTLS, managed identities, retry/ idempotency and transaction status tracking

7. Identity, PKI, MSP and Access Control

Every organization, service and administrative user must have an explicit identity lifecycle. Ledger immutability is ineffective if private keys are poorly controlled.

Control	Development activity	Test / evidence
Organization MSP	Define OEM, battery manufacturer, service/ recycler, platform operator roles	MSP config review; positive/ negative authorization tests
CA hierarchy	Define root/ intermediate/ TLS CA topology and ownership	CA deployment record; chain validation; backup/ restore
Enrollment	Automate certificate enrollment for services and admins	Enrollment test; identity inventory
Renewal	Renew before expiry without service interruption	Short-life certificate test; rolling renewal evidence
Revocation	Revoke compromised users/ services and enforce CRL/ identity update	Revoked identity cannot submit/ endorse
Private keys	Use vault/ HSM where appropriate; no keys in source code/ images	secret scan; key-storage review; rotation test
Role attributes	Bind roles/ tenant/ org to certificate or application policy	Chaincode ABAC/ RBAC negative tests
Admin separation	Separate network admin, chaincode deployer, application writer, auditor	privileged-access review; dual-control evidence

Acceptance principle: a valid Fabric certificate is not sufficient by itself; chaincode and backend policy must confirm that the identity is allowed to perform the requested operation on the specific battery/passport domain.

8. Chaincode / Smart-Contract Design

Keep chaincode deterministic, small and focused on shared business rules. Heavy analytics, AI inference, PDF generation, database joins and external HTTP calls must remain outside chaincode.

Chaincode function	Main checks	State/ event produced
CreatePassport	authorized manufacturer; unique passport ID; mandatory identity fields	passport asset + creation event
AnchorBatch	valid batch ID; root format; time window; event count; duplicate protection	batch commitment + anchor event
AddLifecycleEvent	authorized role; valid event type; permitted state transition	append lifecycle event + current state update
AddComplianceProof	issuer/ role allowed; document hash; version; validity metadata	compliance proof reference
TransferCustody	current custodian approves; new org exists; sequence correct	custody/ ownership state + provenance event
AddCriticalEvent	trusted upstream source; severity/ type schema; evidence hash	thermal/ cyber critical event record
LinkDerivedAsset	valid source modules/ cells; no circular hierarchy; transformation event	parent-child / second-life lineage
VerifyReference	retrieve commitment metadata / root for application verification	read-only query result

Chaincode engineering rules

- Use deterministic logic only; peers must produce identical results.
- Validate every input length/type/format and enforce allowed enum values.
- Use explicit state-transition tables rather than free-form status updates.
- Emit structured chaincode events for backend/RAG/dashboard consumers.
- Version chaincode and data schema; define migration strategy before upgrade.
- Use endorsement policies that reflect who must approve legally/business-critical transitions.
- Include idempotency to make client retries safe.

9. Battery Passport Ledger and Lifecycle State Machine

The ledger should model battery lifecycle transitions explicitly so that invalid sequences cannot be silently accepted.

Lifecycle state	Typical permitted transitions	Required evidence
MANUFACTURED	-> IN_SERVICE, -> REJECTED / SCRAPPED	manufacturing identity, conformity data, initial passport
IN_SERVICE	-> SERVICED, -> RETIRED, -> INCIDENT_HOLD	vehicle/ asset binding, usage summaries, service history
SERVICED	-> IN_SERVICE, -> RETIRED	work order, component replacement, diagnostic proof
INCIDENT_HOLD	-> IN_SERVICE, -> RETIRED, -> QUARANTINED	incident closure, engineering authorization
RETIRED	-> SECOND_LIFE_ASSESSMENT, -> RECYCLING	retirement reason, health/ condition data
SECOND_LIFE_ASSESSMENT	-> SECOND_LIFE, -> RECYCLING	inspection/ test result, grade, derived asset links
SECOND_LIFE	-> SERVICED, -> RECYCLING	new application identity and health history
RECYCLING	-> RECYCLED / CLOSED	collection/ recycler identity, material recovery evidence

State-transition acceptance tests must include valid paths, invalid skips, repeated transitions, stale-version updates and concurrent conflicting transactions.

10. Compliance, Provenance and Event Notarization

The EU Battery Regulation requires a battery passport for covered batteries from 18 February 2027 and Annex XIII defines categories of information to be included. The blockchain layer should not attempt to interpret every regulatory rule in chaincode; it should provide trusted provenance, versioning and evidence references for the application/compliance layer.

Compliance concern	Blockchain contribution	What remains outside chaincode
Battery identity and linkage	immutable identifier and lifecycle lineage	QR/ Digital Link resolution service, UI presentation
Document authenticity	issuer identity + hash + version + timestamp	PDF/ document storage and semantic validation
Carbon footprint / recycled content	versioned declaration proof and provenance	calculation methodology and source datasets
Responsible sourcing	multi-party provenance events and certificate references	supplier due-diligence process and confidential evidence
Performance/ durability data	anchored diagnostic summaries and method versions	raw telemetry, algorithms, detailed calibration
Second-life / recycling	custody, grading, transformation and recovery events	lab reports, process details, commercial terms
Audit history	append-only transaction history and signer identity	auditor workflow, evidence interpretation, legal decision

Recommended interoperability references

- Battery Pass Content Guidance / DIN DKE SPEC 99100 for battery-passport data attributes.
- Battery Pass Technical Guidance for architecture, dynamic data and interoperability considerations.
- GS1 EPCIS 2.0 for interoperable lifecycle/traceability events, including sensor/certification data.
- GS1 Digital Link or equivalent resolver pattern for connecting physical identifiers to digital passport resources.

11. Privacy, Confidentiality and Selective Disclosure

A consortium ledger creates new privacy risks because data replicated to peers can be difficult to retract. Design visibility per field and event before implementation.

Data class	Recommended handling	Verification
Public passport fields	Expose through controlled API/ resolver; ledger may hold state/ hash	public-role access tests; schema validation
Consortium-shared fields	Shared channel or state accessible to approved organizations	org/ role authorization matrix tests
Commercially sensitive fields	Encrypted off-chain + hash; or Fabric private-data collection	unauthorized peer query returns no private value
Personal / driver data	Segregated off-chain; tokenized/ pseudonymized only if truly needed	privacy review; deletion/ retention tests
Security logs / cyber evidence	restricted off-chain evidence vault; ledger hash only	SOC-only access + tamper verification
Cryptographic material	vault/ HSM; never store private keys or secrets on ledger	secret scans, key inventory, HSM/ vault tests

Privacy design rule: hashing personal data does not automatically make it anonymous. Avoid committing direct or reversible identifiers when there is no regulatory/business need.

12. Blockchain-Layer Threat Model and Abuse Cases

Threat / misuse	Potential impact	Primary controls / tests
Unauthorized passport creation	fake batteries/ passports	MSP identity + role policy + chaincode authorization + negative tests
Replay / duplicate anchor	duplicate or conflicting evidence	idempotency key + batch ID uniqueness + replay tests
Off-chain tampering	altered analytics/ document after commitment	Merkle/ hash verification; seeded corruption tests
Compromised application identity	malicious validly signed writes	least privilege, revocation, anomaly monitoring, approval policy
Compromised peer	incorrect/ withheld transactions	endorsement policy, multiple peers/ orgs, block-height reconciliation
Orderer failure	commit outage / backlog	multi-node Raft, failover and recovery tests
Chaincode logic defect	invalid lifecycle state or access	unit tests, static analysis, code review, deterministic test vectors
Private-data leakage	commercial/ privacy breach	PDC policy, channel design, off-chain encryption, unauthorized query tests
Key loss / expiry	service outage / inability to submit	rotation, renewal, backup, HSM/ vault and recovery procedures
DoS / transaction flood	high latency / unavailable passport operations	rate limit, queue, quotas, benchmark under overload
Schema/ version mismatch	unverifiable hashes or broken readers	versioned canonicalization, migration tests, compatibility matrix
Malicious document substitution	fraudulent certification evidence	document hash + issuer identity + versioned reference

13. Verification and Testing Strategy

Testing must prove functional correctness, cryptographic integrity, access control, deterministic smart-contract behavior, multi-organization consensus, resilience and application-level traceability.

Test level	Objective	Methods / evidence
Unit	Prove hash, canonicalization and chaincode functions independently	known-answer SHA-256 vectors, Merkle vectors, state-machine unit tests, authorization tests
Component	Validate peer/ CA/ backend services in isolation	API contract tests, certificate lifecycle, database transactions, event serialization
Chaincode integration	Prove endorsement/ ordering/ commit behavior	Fabric test network, positive/ negative identity and state-transition cases
End-to-end	Prove cloud event -> off-chain store -> root -> ledger -> verification	synthetic telemetry/ passport events and automated proof verification
Security	Prove unauthorized actions and tampering are rejected/ detected	replay, credential misuse, private-data access, altered document/ data, secret scan
Performance	Confirm capacity/ latency headroom	Hyperledger Caliper + backend load test + large ledger/ query tests
Resilience	Confirm recovery from node/ service failure	kill peer/ orderer, restart, network partition simulation, backup/ restore
Compatibility	Confirm upgrades do not break proof verification	schema/ chaincode version matrix, old/ new client regression
Pilot acceptance	Validate operational workflow with real roles and governance	OEM/ fleet/ service user scenarios, audit drill, passport-resolution checks

14. Chaincode, Hashing and Integration Test Catalogue

Test ID	Scenario	Expected result
BC-UT-001	Same canonical event hashed repeatedly	Identical SHA-256 every run/ platform
BC-UT-002	Change one byte/ value in event	Hash differs; Merkle proof fails against original root
BC-UT-003	Reorder JSON object keys before canonicalization	Canonical bytes and hash remain identical
BC-UT-004	Duplicate event_id / batch_id	Chaincode rejects duplicate without changing state
BC-CC-001	Authorized manufacturer creates new passport	Commit succeeds; creation history present
BC-CC-002	Unauthorized service role creates passport	Transaction rejected with reason
BC-CC-003	Invalid lifecycle transition	Transaction rejected; previous state unchanged
BC-CC-004	Required endorser unavailable	Transaction does not commit; backend reports retryable/ failed state
BC-PDC-001	Authorized org queries private data	Private value available
BC-PDC-002	Unauthorized org queries private data	Private value not disclosed
BC-E2E-001	60 s batch -> anchor -> verify random event	Proof validates against ledger root
BC-E2E-002	Modify off-chain record after anchor	Verification reports tamper mismatch
BC-E2E-003	Critical thermal event immediate anchor	Ledger commitment appears within configured critical-event latency
BC-E2E-004	Cyber incident hash from Step 3 trust layer	Incident proof traceable by incident ID without exposing raw log

15. Performance, Scale and Capacity Validation

Because raw telemetry is off-chain, blockchain throughput is dominated by passport/lifecycle events, periodic batch anchors and bursts of critical events. Capacity testing should therefore model both steady anchoring and incident bursts.

Workload	Measure	Test method
Periodic batch anchors	transactions/ s, p50/ p95/ p99 commit latency, failures	Caliper workload with expected fleet batch cadence
Critical event burst	burst throughput and queue time	inject thermal/ cyber event storm above normal peak
Passport creation / transfer	commit latency and endorsement time	multi-org workflow benchmark
Read / verification	ledger query latency + off-chain retrieval + proof verification	API load test with mixed hot/ cold storage
Ledger growth	peer storage, CouchDB index, query performance over time	preload synthetic months/ years of events
Peer/ resource utilization	CPU, RAM, disk I/ O, network	Caliper/ resource monitoring + infrastructure telemetry
Orderer failover	availability, backlog recovery, latency spike	stop leader/ node during load and measure recovery
Private data	dissemination/ commit/ query latency	PDC workload across authorized orgs

Performance design target should be tied to fleet scale. For a 60-second anchor cadence, calculate expected transactions per minute from number of tenants/vehicles/batches, then benchmark at several multiples of forecast peak to establish headroom.

16. Resilience, Recovery and Operations

Operational control	Required implementation	Acceptance evidence
Peer restart / replacement	state sync and ledger catch-up procedure	planned peer loss and rejoin test
Orderer redundancy	multi-node Raft across failure domains	leader/ node failure during active load
CA backup / recovery	protected CA database and key backup / HSM strategy	recovery drill with test identities
Certificate rotation	scheduled renewal before expiry	rolling renewal without transaction outage
Revocation	emergency identity revocation process	compromised identity rejected after policy propagation
Chaincode upgrade	versioned lifecycle approval / rollback plan	old/ new client regression and state migration test
Off-chain backup	database/ object-store backup and point-in-time recovery	restore anchored record and re-verify proof
Reconciliation job	detect missing roots, duplicate batches, ledger/ db mismatch	automated reconciliation report with injected discrepancies
Monitoring	peer/ orderer health, block height, failed tx, queue lag, certificate expiry	dashboard + alert test
Disaster recovery	documented RTO/ RPO and responsibility	full DR exercise and sign-off

17. Acceptance Criteria - Project Targets

The following targets are recommended for the initial controlled production release. Final values should be frozen from actual fleet size, infrastructure sizing, legal obligations and OEM service-level requirements.

Area	Project target	Acceptance method
Hash reproducibility	100% identical digest for deterministic test vectors across supported implementations	cross-language/ platform known-answer test suite
Tamper detection	100% of seeded anchored-record modifications detected	automated off-chain mutation campaign + proof verification
Batch completeness	100% eligible events represented in batch manifest/ root or explicitly rejected with reason	manifest-to-source reconciliation
Duplicate/ replay protection	100% seeded duplicate event/ batch submissions rejected idempotently	retry/ replay injection
Authorization	100% unauthorized role/ org write attempts rejected	RBAC/ ABAC negative test matrix
Private data isolation	0 unauthorized private-data disclosure in test matrix	cross-org PDC query tests
Invalid lifecycle transition	100% rejected without partial state change	state-machine negative tests
Critical vulnerabilities	0 open Critical/ High findings at release	SAST/ SCA/ secret scan + penetration/ security review
Chaincode unit coverage	>= 80% statement/ branch coverage for business-critical chaincode modules	automated test coverage report
Anchor latency	p95 <= 5 s from batch close to committed proof in acceptance environment	timed E2E batch anchor tests
Critical-event anchor	p95 <= 3 s for immediate critical thermal/ cyber proof under nominal load	event-burst test
Capacity headroom	sustain >= 5x forecast peak anchor/ event transaction rate; p95 commit <= 3 s	Caliper benchmark at stepped load
Recovery	RTO <= 30 min and RPO <= 5 min for platform/ off-chain evidence services; ledger committed blocks preserved	documented DR exercise
Traceability	100% sampled ledger commitments resolvable to off-chain source and proof	auditor trace sample
In-scope passport fields	100% configured mandatory data attributes mapped to source, owner, validation and access class	Battery Pass / Annex XIII requirements traceability review

18. Development Lifecycle and Completion Timeline

The baseline below assumes an MVP/engineering pilot with 2-3 developers plus part-time cloud/security/compliance support. Activities can overlap. Production consortium onboarding and legal governance may extend the calendar independently of software development.

Activity	Target timing	Main deliverables	Exit condition
Scope, actors and trust boundary	Week 1	asset/ org list, use cases, on-chain/ off-chain policy	scope and stakeholders approved
Regulatory/ content mapping	Weeks 1-2	Annex XIII/ Battery Pass attribute matrix, privacy classes	in-scope passport data frozen
Event/ data model + canonicalization	Weeks 2-3	schemas, IDs, canonical JSON/ hash spec, batch manifest	known-answer vectors approved
Fabric network / CA / MSP prototype	Weeks 2-4	test network, org identities, channel/ PDC design	multi-org transactions working
Chaincode development	Weeks 4-6	passport, anchor, lifecycle, compliance functions	unit tests and reviews passed
Off-chain store + hashing service	Weeks 4-6	batch scheduler, SHA-256/ Merkle service, proof API	anchor/ verify loop operational
Backend / API integration	Weeks 5-7	Fabric Gateway integration, retry/ status/ idempotency	E2E happy path complete
Passport resolver / UI integration	Weeks 6-8	passport query API, verification status, provenance timeline	user can resolve and verify records
Private data / governance controls	Weeks 7-9	PDC/ RBAC, certificate lifecycle, audit logging	negative access tests pass
Automated V&V; / security tests	Weeks 8-10	unit/ integration/ tamper/ replay/ private-data suite	test report and defect closure
Performance / resilience testing	Weeks 10-11	Caliper report, peer/ orderer failover, recovery evidence	capacity and recovery targets met
Compliance / traceability review	Weeks 11-12	requirements traceability, passport content review	compliance gaps dispositioned
Pilot with representative organizations	Weeks 12-13	pilot passports, operational feedback, governance drill	pilot acceptance
Production acceptance / release	Week 14	release manifest, SBOM, chaincode/ network config, runbooks, sign-offs	G7 production gate passed

19. Acceptance Gates and Sign-Off

Gate	Review question	Required evidence
G0 - Scope Freeze	Are actors, organizations, assets, passport use cases and trust boundaries defined?	architecture, actor/ org matrix, on-chain/ off-chain classification
G1 - Data / Compliance Freeze	Are mandatory passport attributes, lifecycle events, privacy classes and source owners mapped?	requirements matrix, Annex XIII/ Battery Pass mapping, schemas
G2 - Design Freeze	Are Fabric topology, CA/ MSP, channels/ PDCs, endorsement, chaincode and hashing design specified?	HLD/ LLD, interface specs, canonicalization/ hash specification
G3 - Component Ready	Do hashing, chaincode, off-chain store and backend services pass unit/ static/ security checks?	unit reports, SAST/ SCA/ secret scan, chaincode coverage, model/ schema package
G4 - Integrated Validation	Does event -> batch -> Merkle -> Fabric -> verification work for valid/ invalid/ tampered/ replayed cases?	E2E integration, tamper/ replay/ private-data reports
G5 - Security / Performance Acceptance	Are authorization, private data, capacity, failover and recovery targets met?	Caliper report, security test report, DR/ failover evidence
G6 - Pilot Acceptance	Can representative OEM/ fleet/ service/ recycler roles create, transfer, resolve and audit passport evidence correctly?	pilot report, governance/ operations sign-off
G7 - Production Release	Are keys, chaincode/ network lifecycle, monitoring, reconciliation, rollback, SBOM and incident procedures ready?	release manifest, config baselines, runbooks, key inventory, sign-offs

20. Required Engineering Deliverables

Deliverable	Minimum content
Blockchain System Requirements Specification	use cases, actors, integrity/ provenance/ privacy requirements, SLAs, failure handling
Network Architecture Specification	orgs, peers, orderers, CAs/ MSPs, channels, PDCs, endorsement, infrastructure
On-chain / Off-chain Data Specification	data-class decision, retention, ownership, storage references, proof model
Battery Passport Data Model	identity, lifecycle events, compliance references, second-life/ recycling lineage
Canonicalization + Hashing Specification	encoding, ordering, precision, timestamp format, SHA-256/ Merkle algorithm, proof format
Chaincode Design Specification	functions, authorization, state machine, events, error codes, upgrade/ migration
PKI / Identity Operations Guide	enrollment, renewal, revocation, CA/ HSM/ vault, emergency procedures
Backend Integration Specification	Fabric Gateway, APIs, retries, idempotency, transaction tracking, reconciliation
DVP / Test Specification	unit, integration, security, tamper, PDC, performance, resilience, recovery tests
Performance Report	Caliper workload/ config, throughput, latency percentiles, resource use, capacity margin
Security / Privacy Assessment	threat model, role/ access matrix, findings, PII/ on-chain review, vulnerability closure
Traceability Matrix	regulatory/ content requirement -> design -> source -> chaincode/ API -> test evidence
Operations Runbook	monitoring, node/ CA recovery, certificate rotation, chaincode upgrade, reconciliation, incident handling
Release Package	source/ commit IDs, chaincode packages, config, SBOM, test evidence, sign-offs

21. Reference Material, Recent Research and Open Validation Resources

Use the following sources during architecture definition, implementation and validation. Links are included so the team can reproduce the development environment and trace the regulatory/research basis.

Regulation (EU) 2023/1542 - Batteries and waste batteries
Official EU regulation. Annex XIII defines information to be included in the battery passport; passport obligations become relevant from 18 February 2027 for covered batteries.
https://eur-lex.europa.eu/eli/reg/2023/1542/oj

Battery Pass Technical Guidance (2024)
Industry technical guidance for EU Battery Passport implementation, architecture, dynamic data and interoperability.
https://thebatterypass.eu/assets/images/technical-guidance/pdf/2024_BatteryPassport_Technical_Guidance.pdf

Battery Pass Content Guidance / DIN DKE SPEC 99100
Battery-passport data attribute guidance and implementation material.
https://thebatterypass.eu/battery-pass/materials/

Global Battery Alliance - 2024 Battery Passport Pilots
Real-world multi-consortium passport pilot results and implementation lessons.
https://www.globalbattery.org/media/publications/gba-batterypassport-2024-v1-web.pdf

GS1 EPCIS / CBV 2.0
Interoperable traceability events; supports sensor/certification data and REST/JSON-LD interfaces.
https://www.gs1.org/standards/epcis

Hyperledger Fabric 2.5 - Test Network
Open-source local multi-org network for chaincode/application testing.
https://hyperledger-fabric.readthedocs.io/en/release-2.5/test_network.html

Hyperledger Fabric - Chaincode deployment/lifecycle
Official smart-contract deployment and lifecycle guidance.
https://hyperledger-fabric.readthedocs.io/en/release-2.5/deploy_chaincode.html

Hyperledger Fabric - Private Data
Official private-data collection concepts/tutorials for restricted consortium data.
https://hyperledger-fabric.readthedocs.io/en/release-2.5/private-data/private-data.html

Hyperledger Fabric CA
Open-source identity enrollment, renewal and revocation tooling.
https://hyperledger-fabric-ca.readthedocs.io/en/latest/users-guide.html

Hyperledger Caliper
Open-source blockchain benchmark framework for throughput, latency and resource metrics.
https://hyperledger-caliper.github.io/caliper/0.7.1/tutorial/fabric/

ISO 22739:2024
Blockchain and distributed ledger technologies - Vocabulary.
https://www.iso.org/standard/82208.html

ISO 23257:2022
Blockchain and distributed ledger technologies - Reference architecture.
https://www.iso.org/standard/75093.html

ISO/TR 23244:2020
Privacy and personally identifiable information protection considerations for blockchain/DLT.
https://www.iso.org/standard/75061.html

Terke■ et al., 2024 - IEEE Access
Battery Passport for Second-Life Batteries: Potential Applications and Challenges. DOI 10.1109/ACCESS.2024.3450790.
https://doi.org/10.1109/ACCESS.2024.3450790

Butera & Gatteschi, 2024 - IEEE BRAINS
EU Battery Regulation and the Role of Blockchain in Creating Digital Battery Passports. DOI 10.1109/BRAINS63024.2024.10732126.
https://doi.org/10.1109/BRAINS63024.2024.10732126

Neri et al., 2024 - IFAC-PapersOnLine
Distributed Ledger Technology selection for Digital Battery Passport: A BWM-TOPSIS approach.
https://doi.org/10.1016/j.ifacol.2024.09.258

Hasan et al., 2025 - Sustainable Futures
Using composable NFTs and blockchain for the creation of EV battery digital passports with sustainability and traceability features.
https://doi.org/10.1016/j.sftr.2025.100847

BlockBaRT, 2025
Blockchain-based Battery Passport with selective disclosure of verifiable data; useful for privacy-oriented research direction.

https://saicsit2025.org.za/FDCR_2025_Final.pdf

22. Open-Source Implementation and Validation Path

A practical team can reproduce the core Step 6 workflow without proprietary blockchain software.

Stage	Open-source resource	What to validate
1. Local network	hyperledger/ fabric-samples test-network	multi-org peer/ orderer/ channel basics
2. Smart contracts	asset-transfer-basic + custom battery-passport chaincode	deployment lifecycle, deterministic CRUD/ state transitions
3. Identities	Hyperledger Fabric CA	enrollment, role identities, renewal, revocation
4. Private data	Fabric private-data sample / collections	authorized vs unauthorized read/ write behavior
5. Hash proof service	Python/ Node standard SHA-256 + Merkle implementation	known-answer vectors, inclusion proof, one-bit tamper failure
6. Off-chain store	PostgreSQL / TimescaleDB + object storage	event manifest, retention, source-to-proof resolution
7. Integration	Fabric Gateway SDK + FastAPI backend	submit/ evaluate, retries, status, idempotency
8. Performance	Hyperledger Caliper	throughput, commit latency, resource use, stepped-load capacity
9. Traceability format	GS1 EPCIS 2.0 schemas/ sandbox	interoperable lifecycle event capture/ query
10. Battery passport content	Battery Pass demonstrator / guidance + GBA pilots	required attributes, role visibility, lifecycle examples

Recommended validation dataset generation

- Generate synthetic battery lifecycle records from manufacturing through in-service, service, second-life assessment and recycling.
- Generate 60-second telemetry-summary batches with deterministic event IDs and timestamps.
- Seed deliberate off-chain tampering, duplicate events, stale versions, invalid state transitions and unauthorized organization identities.
- Generate critical thermal/cyber events from the Step 5/Step 3 pipeline and validate immediate anchoring plus later proof resolution.
- Retain test vectors and ledger snapshots so regression tests can be repeated after chaincode/schema upgrades.

Minimum production-release package

- Approved requirements and architecture
- Versioned chaincode packages and source commit IDs
- Network / CA / MSP / channel / PDC configuration baseline
- On-chain/off-chain data dictionary and canonicalization spec
- Automated test suite and acceptance report
- Caliper capacity report
- Security/privacy review and vulnerability closure
- Battery-passport requirements traceability matrix
- Operations/DR/key-rotation/reconciliation runbooks
- SBOM, release manifest and signed G0-G7 approvals

Outcome: Step 6 is accepted only when the organization can independently reproduce a proof: locate a passport/event, retrieve the governed off-chain evidence, recompute its cryptographic commitment, verify it against the Fabric ledger, identify the responsible organization and version, and demonstrate that unauthorized modification or access is prevented or detected.